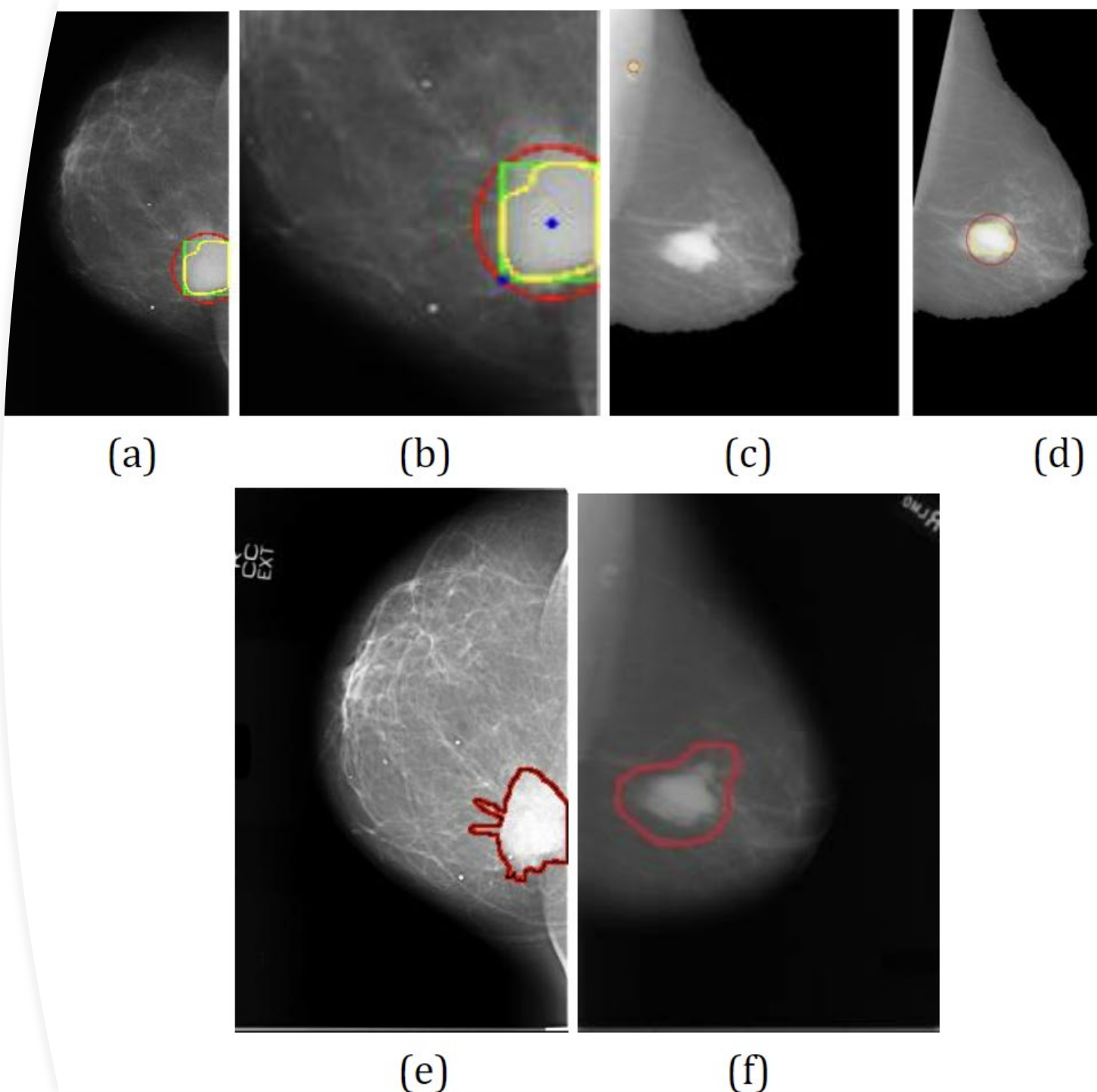
Problem Statement (1)

- Diagnosing a breast tumor as cancerous (Malignant) or not (Benign) correctly makes a big difference in life expectancy and quality of life of a patient.
- Misdiagnosis of a breast tumor can have drastic consequences on the medical condition of a patient.
- In many cases, quite frequently misdiagnosis of breast tumors happen in the presence of human intervention and medical experts.
- So, it means that sometimes even medical experts are not able to diagnose breast tumors correctly.
- Hence, it becomes very important to develop a Machine Learning based system which can diagnose breast tumors correctly at a very high rate, possibly even higher than medical experts.
- So, given a dataset of breast tumors we must develop a system which will learn to perform correct diagnosis of breast tumors from the same dataset.

Problem Statement (2)

- In order to solve this problem, we will using the Wisconsin Breast Cancer Tumor dataset to train a Naive Bayes Classifier from scratch, to classify the tumors as either Benign or Malignant and the evaluating it's performance.



Exploratory Data Analysis (EDA)

Step 1: Import the necessary libraries

```
import pandas as pd
# For loading data as Pandas Dataframe
import numpy as np
# For performing Matrix Calculations
import seaborn as sns
# For visualizing statistical results
import scipy.stats as s
# For computing Statistical Functions
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from matplotlib import cm
#For plotting Distributions
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
# For evaluating the performance of the model
```

Step 2: Read the dataset

- **Download the dataset from the following link:**
- https://drive.google.com/file/d/1JCgSmjL1AdssrFmZpGDJJVEhXSp72U8Q/view?usp=sharing

```
data = pd.read_csv("BreastCancerData.csv")
```

Step 3: Check the dataset

```
data
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	...
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	0.14710	...
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	0.07017	...
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	0.12790	...
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	0.10520	...
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.10430	...
...	...	...	...	...	...	...	...	...	...	...	...
564	926424	M	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	...
565	926682	M	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	...
566	926954	M	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	...
567	927241	M	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	...
568	92751	B	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	...

569 rows × 33 columns

Check detailly for the data

- As can be seen that the dataset has 33 columns and since, the column having name "Unnamed: 32" has all the missing values therefore first of all we will get rid of that column.

_worst	perimeter_worst	area_worst	smoothness_worst	compactness_worst	concavity_worst	concave points_worst	symmetry_worst	fractal_dimension_worst	Unnamed: 32
17.33	184.60	2019.0	0.16220	0.66560	0.7119	0.2654	0.4601	0.11890	NaN
23.41	158.80	1956.0	0.12380	0.18660	0.2416	0.1860	0.2750	0.08902	NaN
25.53	152.50	1709.0	0.14440	0.42450	0.4504	0.2430	0.3613	0.08758	NaN
26.50	98.87	567.7	0.20980	0.86630	0.6869	0.2575	0.6638	0.17300	NaN
16.67	152.20	1575.0	0.13740	0.20500	0.4000	0.1625	0.2364	0.07678	NaN
...	...	...	...	...	...	...	...	...	...
26.40	166.10	2027.0	0.14100	0.21130	0.4107	0.2216	0.2060	0.07115	NaN
38.25	155.00	1731.0	0.11660	0.19220	0.3215	0.1628	0.2572	0.06637	NaN
34.12	126.70	1124.0	0.11390	0.30940	0.3403	0.1418	0.2218	0.07820	NaN
39.42	184.60	1821.0	0.16500	0.86810	0.9387	0.2650	0.4087	0.12400	NaN
30.37	59.16	268.6	0.08996	0.06444	0.0000	0.0000	0.2871	0.07039	NaN

```
data_columns = data.columns
```

```
data_columns
```

```
Index(['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
       'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
       'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
       'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
       'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
       'fractal_dimension_se', 'radius_worst', 'texture_worst',
       'perimeter_worst', 'area_worst', 'smoothness_worst',
       'compactness_worst', 'concavity_worst', 'concave points_worst',
       'symmetry_worst', 'fractal_dimension_worst', 'Unnamed: 32'],
      dtype='object')
```

```
len(data_columns)
```

```
33
```

Step 4: Removing error column/s

Removing the "Unnamed : 32" column from dataset.

```
data.drop(labels=data_columns[32],axis=1,inplace=True)
```

```
data
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	...
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	0.14710	...
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	0.07017	...
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	0.12790	...
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	0.10520	...
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.10430	...
...	...	...	...	...	...	...	...	...	...	...	...
564	926424	M	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	...
565	926682	M	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	...
566	926954	M	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	...
567	927241	M	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	...
568	92751	B	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	...

569 rows × 32 columns

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 32 columns):
 #   Column                   Non-Null Count  Dtype  
---  ------                   --------------  -----  
 0   id                       569 non-null    int64  
 1   diagnosis                569 non-null    object 
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
 22  radius_worst             569 non-null    float64
 23  texture_worst            569 non-null    float64
 24  perimeter_worst          569 non-null    float64
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
dtypes: float64(30), int64(1), object(1)
memory usage: 142.4+ KB
```

Step 5: Check statistical information

- **One very valuable information that we can draw from this statistical information is that majority of the features in our dataset is floating point type except for 'id' and 'diagnosis' features.**

Step 6: Check the missing values

- It was very easy for us to determine through visual inspection that "Unnamed : 32" feature has NaN values but it's quite difficult to determine that any other features has some missing values or not just by sheer visual inspection so let's try to determine that whether any other feature has missing values or not.
- As it can be observed that none of the other features has missing values so we don't have to probably drop any other feature now.

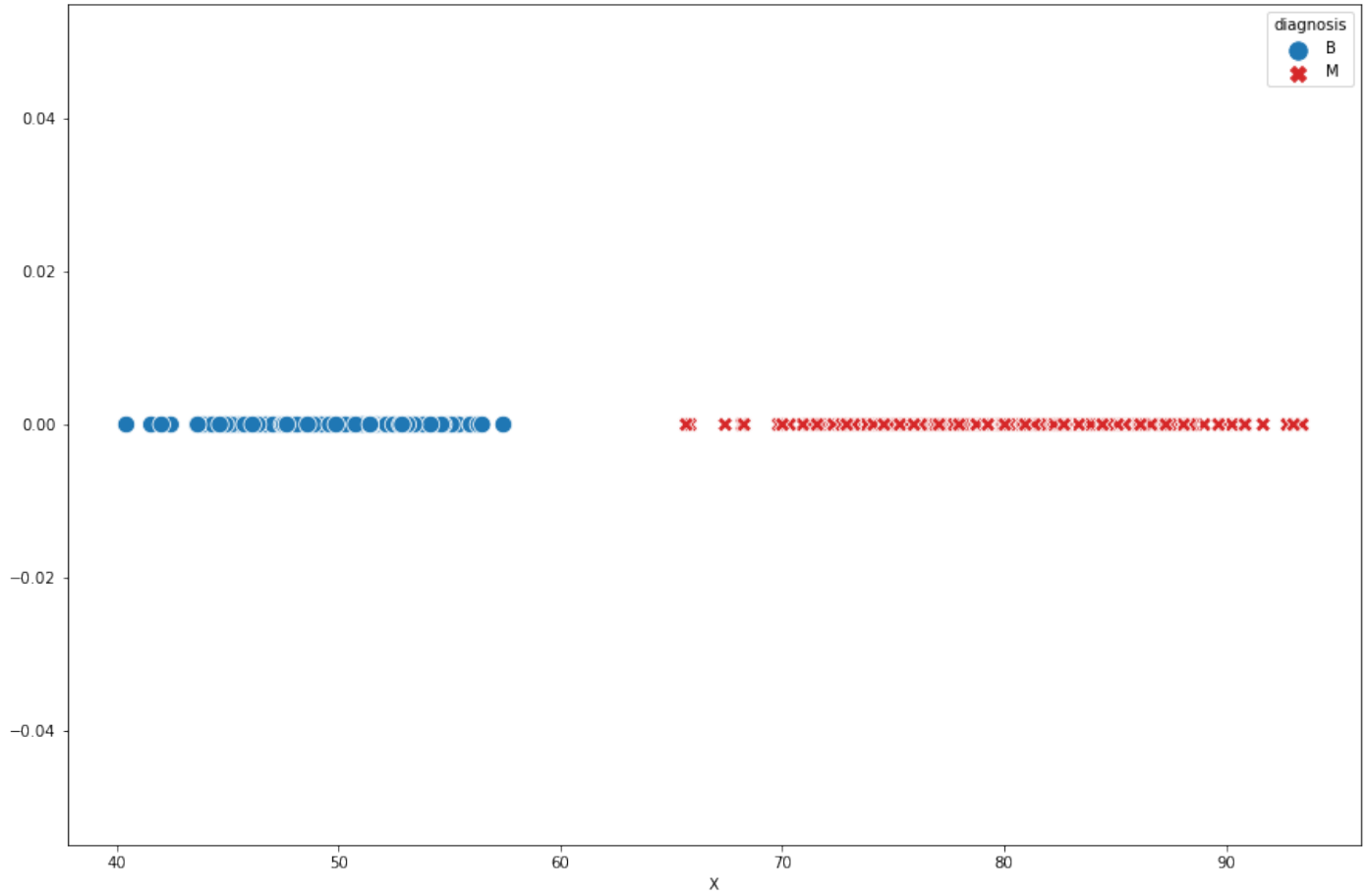
```
data.isnull().sum().sort_values(ascending=False)
```

```
id                         0
diagnosis                  0
symmetry_worst             0
concave points_worst       0
concavity_worst            0
compactness_worst          0
smoothness_worst           0
area_worst                 0
perimeter_worst            0
texture_worst              0
radius_worst               0
fractal_dimension_se       0
symmetry_se                0
concave points_se          0
concavity_se               0
compactness_se             0
smoothness_se              0
area_se                    0
perimeter_se               0
texture_se                 0
radius_se                  0
fractal_dimension_mean     0
symmetry_mean              0
concave points_mean        0
concavity_mean             0
compactness_mean           0
smoothness_mean            0
area_mean                  0
perimeter_mean             0
texture_mean               0
radius_mean                0
fractal_dimension_worst    0
dtype: int64
```

THE CONCEPT

Data quality issues ➔ Example for ideal data

- In order to perform as much best classification as possible, in ideal scenario the data should look something like in the figure.
- Why is it called ideal data??
  There is a huge gap between blue data points and red data points.



THE CONCEPT

Ideal Distributions

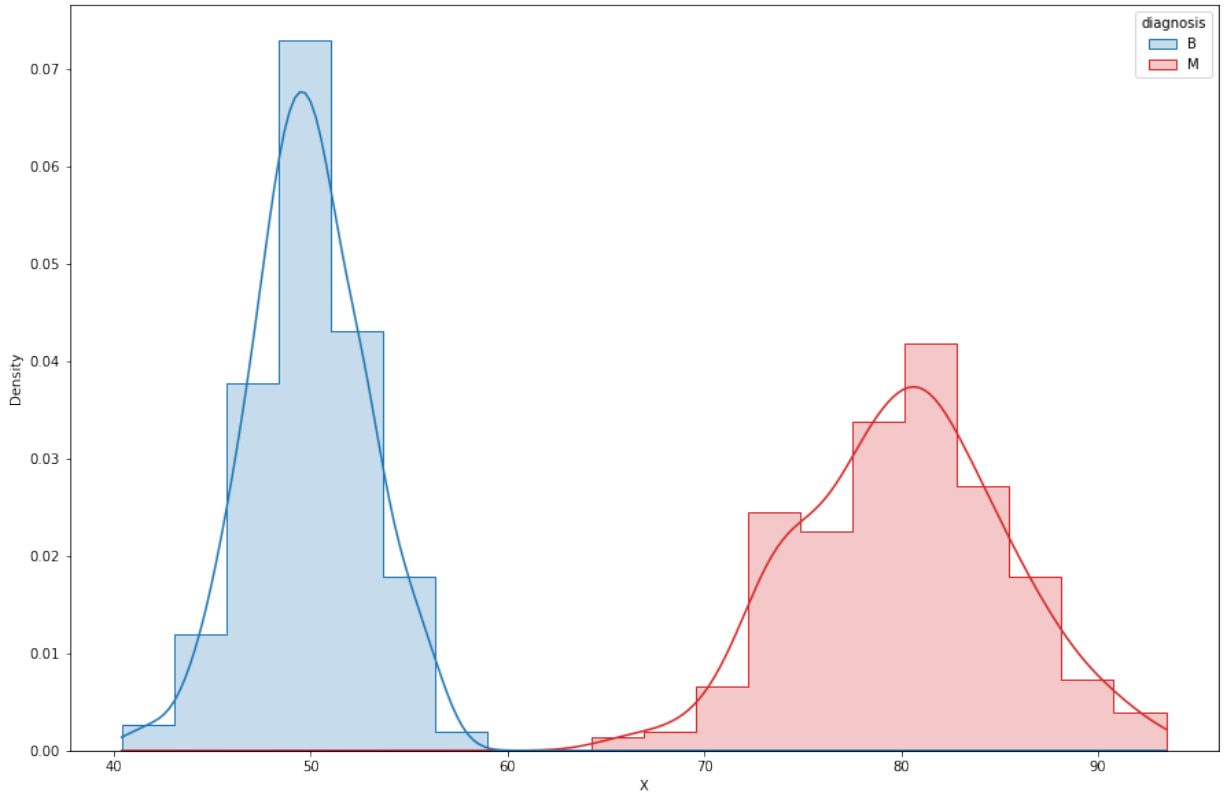
$$P(X|Class = Benign)$$

$$P(X|Class = Malignant)$$

Normal Probability Density Function

$$F(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/2\sigma^2}$$

- Ideal distributions ➔ two distribution classes should not overlap each other ➔ it will result 100% of accuracy.
- **In other words, it can be said that feature X has strong classification power.**



Step 7: Plot the distribution function from the dataset

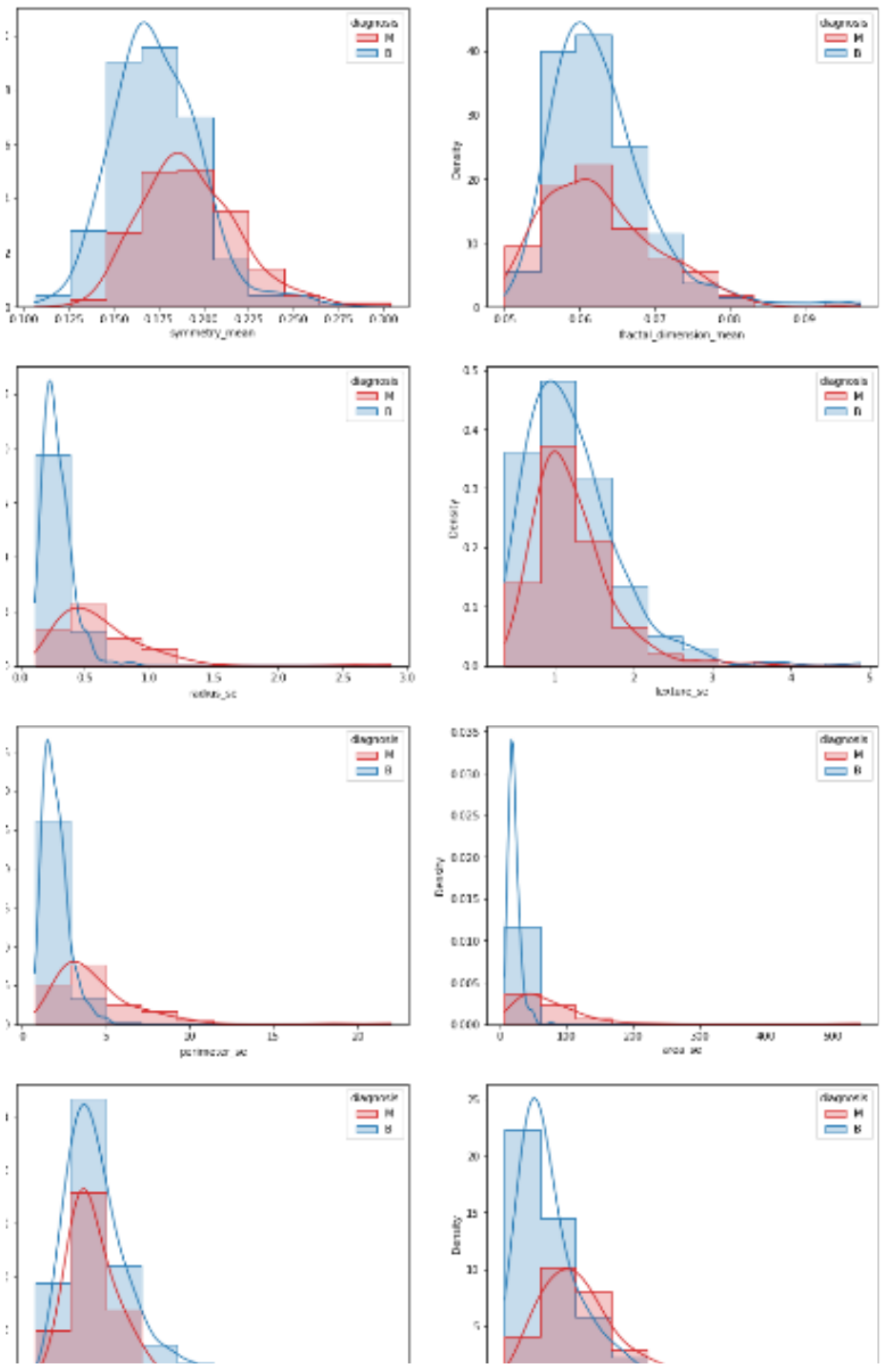
- **Plot $P(X|Class = Benign)$ and $P(X|Class = Malignant)$ for different features one by one for X equals to different features in the data_columns list**

```
def plot_grid_histplot(data,data_columns,shape,figure_size):

    data_columns = np.array(data_columns).reshape(shape[0],shape[1])
    fig, axes = plt.subplots(shape[0],shape[1],figsize=figure_size)

    for i in range(data_columns.shape[0]):
        for j in range(data_columns.shape[1]):

            sns.histplot(data=data,x=data_columns[i,j],hue='diagnosis',stat='density',bins=10,kde=True,
                palette=[sns.color_palette()[3],sns.color_palette()[0]],element='step',ax=axes[i,j])
```

```
plot_grid_histplot(data,data_columns,shape=(15,2),figure_size=(15,95))
```

Results of distribution function

- Distribution functions of each feature are different from ideal scenario because those have overlap area between two classes for every single feature.
- **Check the feature that have overlapping as minimum as possible!**
- **Find 8 features that have minimum overlapping area!**



Select appropriate features

- **If seen above then the for the following features, two probability distributions are overlapping as minimum as possible:**

a) radius_mean

b) perimeter_mean

c) area_mean

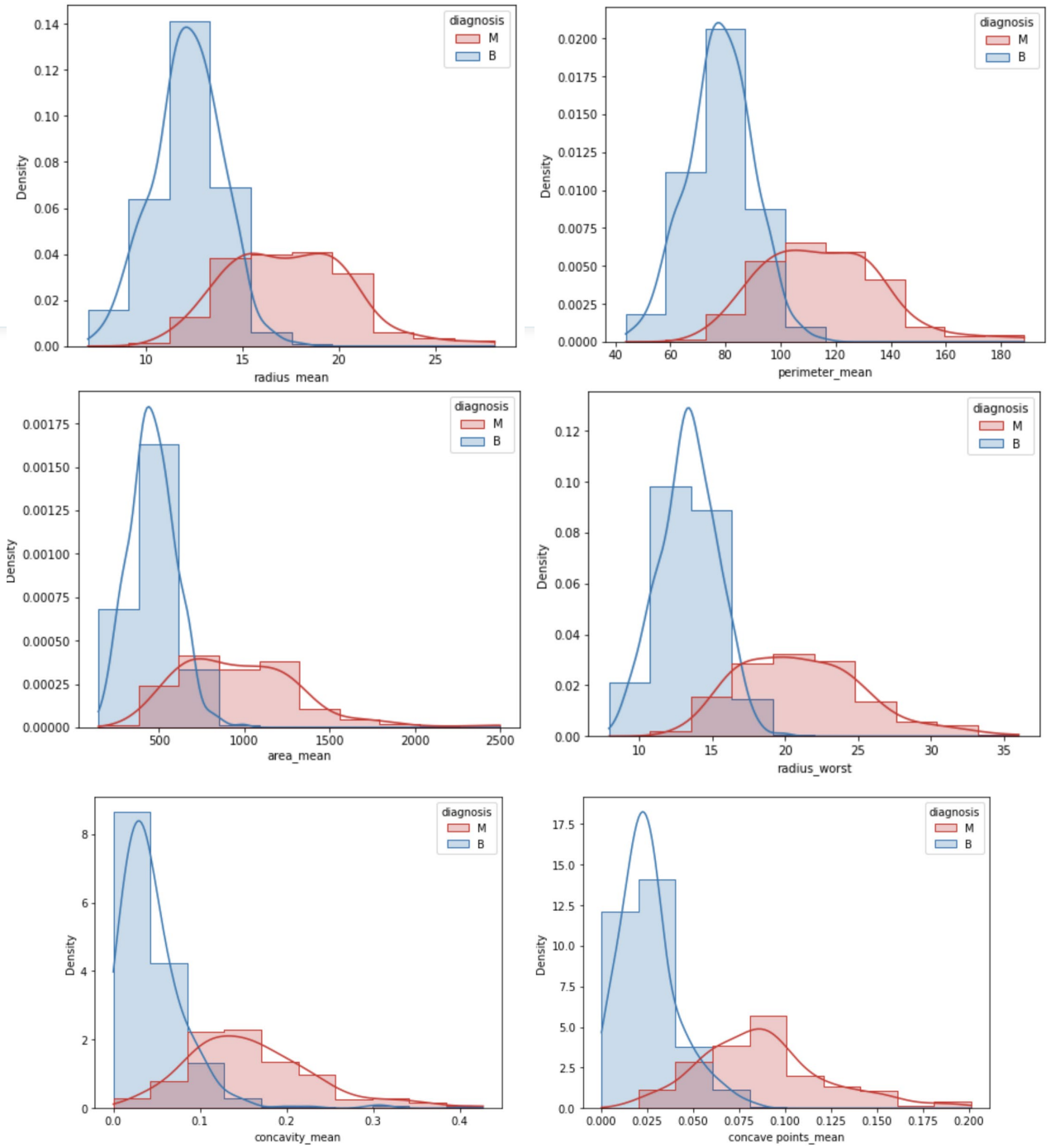
d) concavity_mean

e) concave_points_mean

f) radius_worst

g) perimeter_worst

h) area_worst



Step 8: Verify appropriate features using heatmap (1)

```
data_copy = data.replace(to_replace=['B','M'],value=[0,1],inplace=False)
```

- **Let's first observe the values behind this heatmap:**

```
corr_df = data_copy.corr()
```

```
corr_df
```

Step 8: Verify appropriate features using heatmap (2)

- All the values are ranging between -1 to 1.
- Each individual value is the value of Pearson correlation coefficient between two features.

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

Where,

r = Pearson Correlation Coefficient

x_i = x variable samples

y_i = y variable sample

$\bar{x}$ = mean of values in x variable

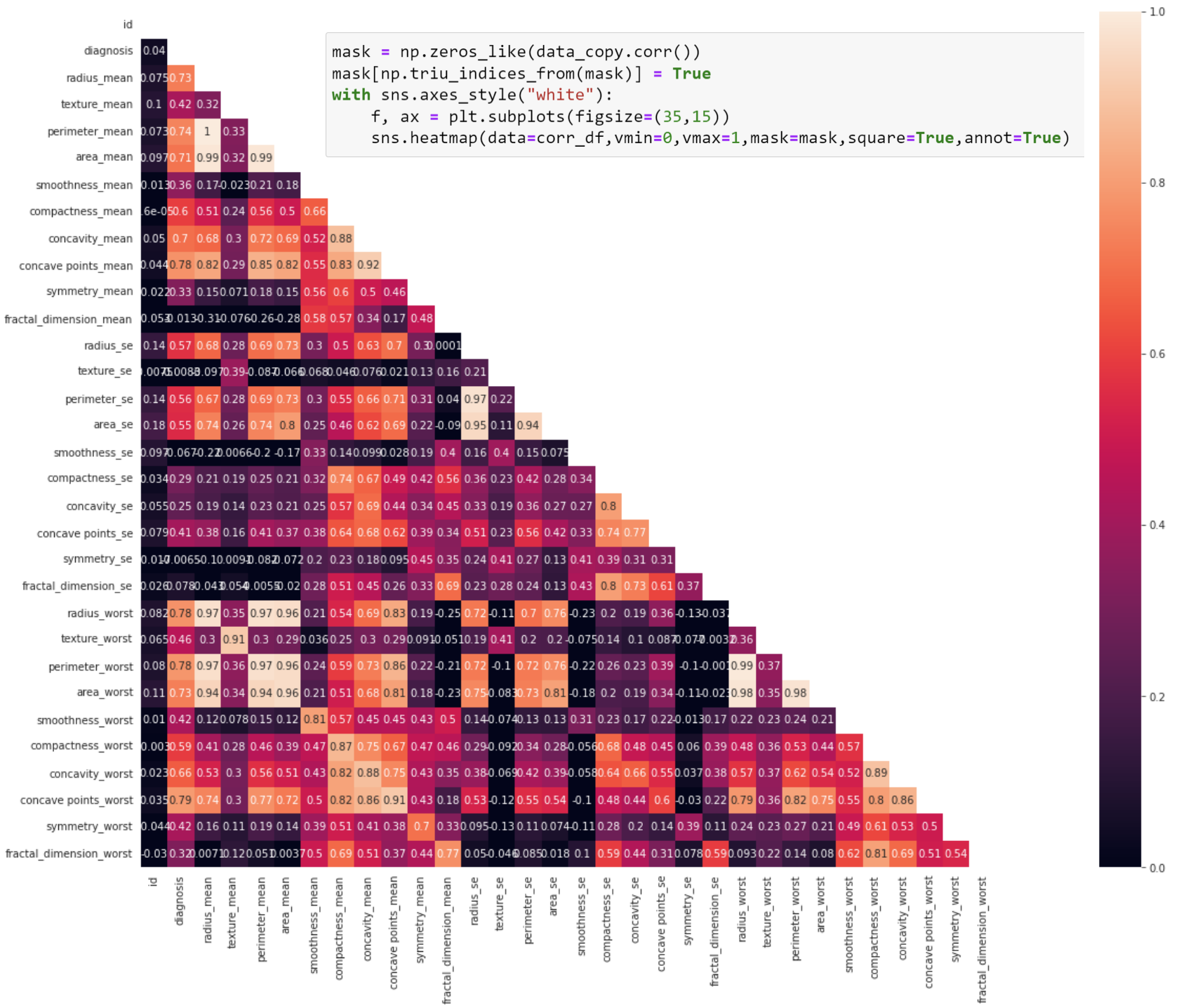
$\bar{y}$ =mean of values in y variable

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	sm
id	1.000000	0.039769	0.074626	0.099770	0.073159	0.096893	
diagnosis	0.039769	1.000000	0.730029	0.415185	0.742636	0.708984	
radius_mean	0.074626	0.730029	1.000000	0.323782	0.997855	0.987357	
texture_mean	0.099770	0.415185	0.323782	1.000000	0.329533	0.321086	
perimeter_mean	0.073159	0.742636	0.997855	0.329533	1.000000	0.986507	
area_mean	0.096893	0.708984	0.987357	0.321086	0.986507	1.000000	
smoothness_mean	-0.012968	0.358560	0.170581	-0.023389	0.207278	0.177028	
compactness_mean	0.000096	0.596534	0.506124	0.236702	0.556936	0.498502	
concavity_mean	0.050080	0.696360	0.676764	0.302418	0.716136	0.685983	
concave points_mean	0.044158	0.776614	0.822529	0.293464	0.850977	0.823269	
symmetry_mean	-0.022114	0.330499	0.147741	0.071401	0.183027	0.151293	
fractal_dimension_mean	-0.052511	-0.012838	-0.311631	-0.076437	-0.261477	-0.283110	
radius_se	0.143048	0.567134	0.679090	0.275869	0.691765	0.732562	
texture_se	-0.007526	-0.008303	-0.097317	0.386358	-0.086761	-0.066280	
perimeter_se	0.137331	0.556141	0.674172	0.281673	0.693135	0.726628	
area_se	0.177742	0.548236	0.735864	0.259845	0.744983	0.800086	
smoothness_se	0.096781	-0.067016	-0.222600	0.006614	-0.202694	-0.166777	
compactness_se	0.033961	0.292999	0.206000	0.191975	0.250744	0.212583	
concavity_se	0.055239	0.253730	0.194204	0.143293	0.228082	0.207660	
concave points_se	0.078768	0.408042	0.376169	0.163851	0.407217	0.372320	
symmetry_se	-0.017306	-0.006522	-0.104321	0.009127	-0.081629	-0.072497	
fractal_dimension_se	0.025725	0.077972	-0.042641	0.054458	-0.005523	-0.019887	
radius_worst	0.082405	0.776454	0.969539	0.352573	0.969476	0.962746	
texture_worst	0.064720	0.456903	0.297008	0.912045	0.303038	0.287489	
perimeter_worst	0.079986	0.782914	0.965137	0.358040	0.970387	0.959120	

Heatmap

- Upper triangle = lower triangle values.
- **Darker the color in a cell of the heatmap, weaker the relationship between the two features located at the row and column whose intersection is resulting in that cell position. Brighter the color in a cell, stronger the relationship.**

```
mask = np.zeros_like(data_copy.corr())
mask[np.triu_indices_from(mask)] = True
with sns.axes_style("white"):
    f, ax = plt.subplots(figsize=(35,15))
    sns.heatmap(data=corr_df,vmin=0,vmax=1,mask=mask,square=True,annot=True)
```



Pearson correlation function (1)

- **Pearson Correlation Coefficient is computed as:**

$$r_{X_1,X_2} = \frac{Cov(X_1, X2)}{\sigma X1 \cdot \sigma X_2}$$

σ_1 = Standard Deviation of feature X_1

σ_2 = Standard Deviation of feature X_2

Where r_{X_1,X_2} = Pearson Correlation Coefficient

$Cov(X_1, X_2)$ = Covariance between features X_1 and X_2 which is given by:

$$Cov(X_1, X2) = \frac{\sum i = 1^{N}(x1^{i} - \mu X_1) \cdot (x2^{i} - \mu X_2)}{N}$$

Pearson correlation function (2)

- **Property regarding Pearson Correlation Coefficient is that:**

$$-1 \leq r_{X_m,X_n} \leq +1$$

- **Where the value of -1 imples that there is strong negative relationship between features *Xm* and *Xn* and on the other hand the value of +1 implies that there is a strong positive relationship between them. Whereas, the value close to 0 irrespective of the sign implies that there is no relationship between the two features.**
- r_{X_m,X_n} **is sometimes also called normalized covariance.**

Pearson correlation function (3)

- If carefully observed, then the features which were having a good classification power are also the ones which have high magnitude of pearson correlation coefficient with the 'diagnosis' feature so it can be said that they are the ones on whom the decision of a tumor being classified as Benign or Malignant strongly depends.
- Select the features which have strong relationship with 'diagnosis' feature :

```
concave points_worst    0.793566
perimeter_worst         0.782914
concave points_mean     0.776614
radius_worst            0.776454
perimeter_mean          0.742636
area_worst              0.733825
radius_mean             0.730029
area_mean               0.708984
Name: diagnosis, dtype: float64
```

```
strong_relation_features = pd.Series(corr_df['diagnosis']).nlargest(n=9).iloc[1:]
```

```
strong_relation_features
```

Step 9: Remove uncorrelated features (1)

- **Therefore, the top eight features will be selected in our dataset and rest of the features will be discarded from our dataset. Now, let's select these top eight features from our dataset along with the 'diagnosis' column and let's observe the heatmap of them once again.**

```
diagnosis = data_copy['diagnosis']
data_copy = data_copy[list(strong_relation_features.to_dict().keys())]
```

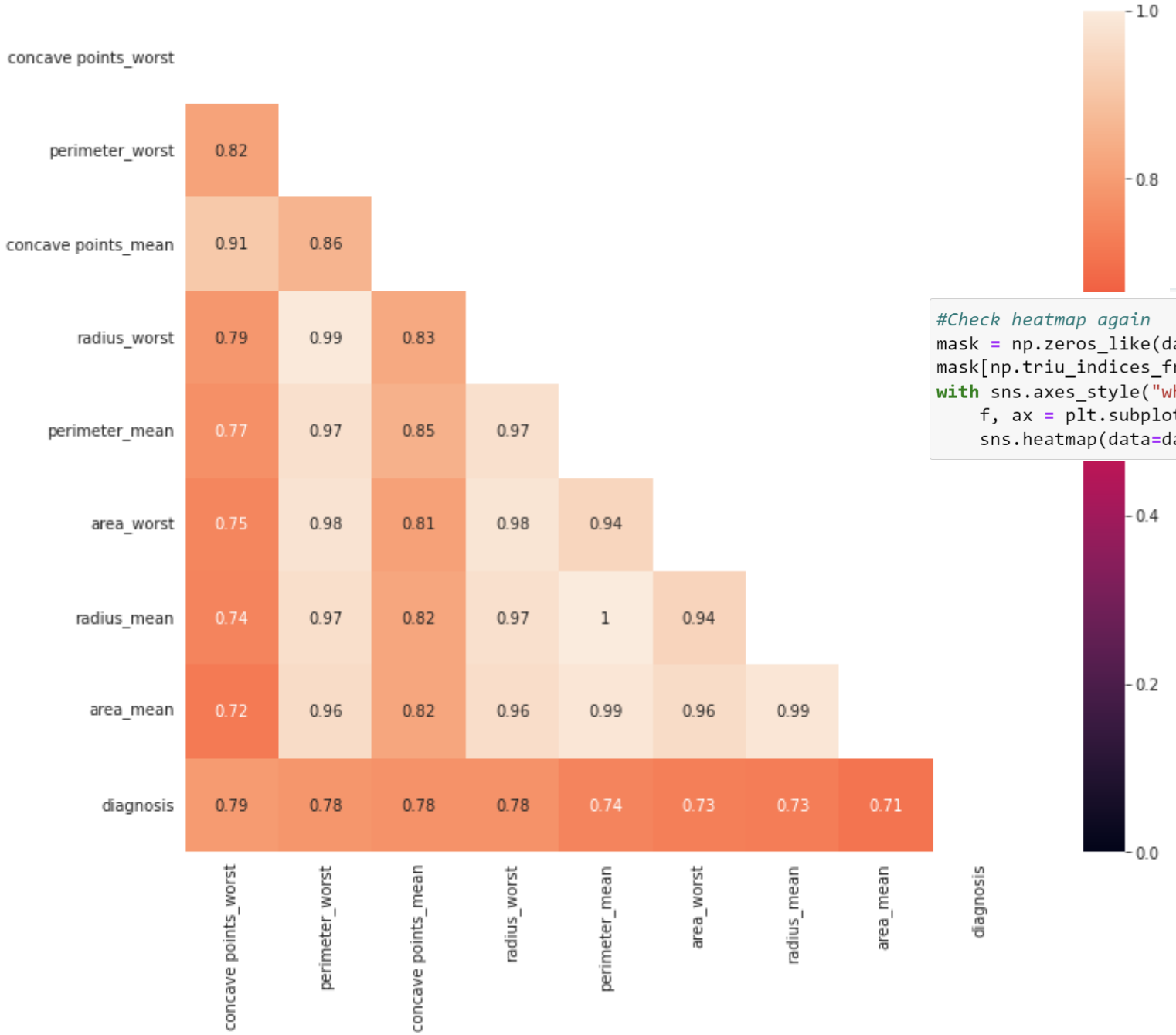
```
data_copy['diagnosis'] = diagnosis
```

```
data_copy
```

	concave points_worst	perimeter_worst	concave points_mean	radius_worst	perimeter_mean	area_worst	radius_mean	area_mean	diagnosis
0	0.2654	184.60	0.14710	25.380	122.80	2019.0	17.99	1001.0	1
1	0.1860	158.80	0.07017	24.990	132.90	1956.0	20.57	1326.0	1
2	0.2430	152.50	0.12790	23.570	130.00	1709.0	19.69	1203.0	1
3	0.2575	98.87	0.10520	14.910	77.58	567.7	11.42	386.1	1
4	0.1625	152.20	0.10430	22.540	135.10	1575.0	20.29	1297.0	1
...	...	...	...	...	...	...	...	...	...
564	0.2216	166.10	0.13890	25.450	142.00	2027.0	21.56	1479.0	1
565	0.1628	155.00	0.09791	23.690	131.20	1731.0	20.13	1261.0	1
566	0.1418	126.70	0.05302	18.980	108.30	1124.0	16.60	858.1	1
567	0.2650	184.60	0.15200	25.740	140.10	1821.0	20.60	1265.0	1
568	0.0000	59.16	0.00000	9.456	47.92	268.6	7.76	181.0	0

569 rows × 9 columns

Step 9: Remove uncorrelated features (2)




```
#Check heatmap again
mask = np.zeros_like(data_copy.corr())
mask[np.triu_indices_from(mask)] = True
with sns.axes_style("white"):
    f, ax = plt.subplots(figsize=(13,10))
    sns.heatmap(data=data_copy.corr(),vmin=0,vmax=1,mask=mask,square=True,annot=True)
```

- Check heatmap using 8 top correlated features.

Step 10: Check singular matrix is singular or not (1)

- Another interesting thing we can observe here is that definitely these top eight features have a good correlation with diagnosis but these top eight features have even higher correlation among themselves also. And if we compute the covariance matrix or correlation matrix in this case then probably, our covariance matrix is going to be singular.
- Let's try to check that whether the covariance matrix will be singular or not.

```
#check singular matrix be singular or not
data_copy_cov = np.array(data_copy[list(strong_relation_features.to_dict().keys())].cov())
```

```
data_copy_cov
```

Step 10: Check singular matrix is singular or not (2)

- Covariance Matrix Results :

```
data_copy_cov
```

```
array([[4.32074068e-03, 1.80307085e+00, 2.32144382e-03, 2.50164782e-01,
        1.23184809e+00, 2.79722743e+01, 1.72392739e-01, 1.67017894e+01],
       [1.80307085e+00, 1.12913085e+03, 1.11601611e+00, 1.61387312e+02,
        7.92328208e+02, 1.87028700e+04, 1.14288570e+02, 1.13417898e+04],
       [2.32144382e-03, 1.11601611e+00, 1.50566077e-03, 1.55720694e-01,
        8.02360375e-01, 1.78868809e+01, 1.12475116e-01, 1.12419582e+01],
       [2.50164782e-01, 1.61387312e+02, 1.55720694e-01, 2.33602242e+01,
        1.13858063e+02, 2.70785053e+03, 1.65137495e+01, 1.63752134e+03],
       [1.23184809e+00, 7.92328208e+02, 8.02360375e-01, 1.13858063e+02,
        5.90440480e+02, 1.30261484e+04, 8.54471417e+01, 8.43577235e+03],
       [2.79722743e+01, 1.87028700e+04, 1.78868809e+01, 2.70785053e+03,
        1.30261484e+04, 3.24167385e+05, 1.88822722e+03, 1.92192558e+05],
       [1.72392739e-01, 1.14288570e+02, 1.12475116e-01, 1.65137495e+01,
        8.54471417e+01, 1.88822722e+03, 1.24189201e+01, 1.22448341e+03],
       [1.67017894e+01, 1.13417898e+04, 1.12419582e+01, 1.63752134e+03,
        8.43577235e+03, 1.92192558e+05, 1.22448341e+03, 1.23843554e+05]])
```

```
data_copy_cov.shape
```

```
(8, 8)
```

Step 10: Check singular matrix is singular or not (3)

- Let's see whether this matrix is singular or not. So, in order to check whether it is singular or not, we should check it's determinant because the determinant of the singular matrix is always zero.

```
#Calculate determinant from the matrix
data_copy_cov_det = np.linalg.det(data_copy_cov)
```

```
data_copy_cov_det
```

```
114.15727331084628
```

- The result shows that the matrix is not singular.

Step 11: Calculate the number of each class

- **Benign tumors (negative class, 0 class)**

```
#how many benign tumors (negative class, 0 class) are there in our data ?
data_copy[data_copy['diagnosis'] == 0].shape[0]
```

```
357
```

- **Malignant tumors (positive class, 1 class)**

```
#how many malignant tumors (positive class, 1 class) are there in our data ?
data_copy[data_copy['diagnosis'] == 1].shape[0]
```

```
212
```

Step 12: Split into training and Cross Validation Data

- Example: Split 75% of benign and malignant tumors in training data, 25% into cross validation data.

```
class0_data = data_copy[data_copy['diagnosis'] == 0]
class1_data = data_copy[data_copy['diagnosis'] == 1]

class0_training_data = class0_data.iloc[0:int(0.75*len(class0_data))]
class1_training_data = class1_data.iloc[0:int(0.75*len(class1_data))]

class0_cv_data = class0_data.iloc[int(0.75*len(class0_data)):]
class1_cv_data = class1_data.iloc[int(0.75*len(class1_data)):]

training_data = pd.concat([class0_training_data,class1_training_data]) #sum of training data from class 0 and 1
cv_data = pd.concat([class0_cv_data,class1_cv_data])
```

```
training_data
```

	concave points_worst	perimeter_worst	concave points_mean	radius_worst	perimeter_mean	area_worst	radius_mean	a
19	0.12880	99.70	0.047810	15.110	87.46	711.2	13.540	
20	0.07283	96.09	0.031100	14.500	85.63	630.5	13.080	
21	0.06227	65.13	0.020760	10.230	60.34	314.9	9.504	
37	0.05013	84.46	0.029230	13.300	82.61	545.9	13.030	
46	0.02564	57.26	0.005917	8.964	51.71	242.2	8.196	
...	...	...	...	...	...	...	...	
335	0.18270	143.20	0.099340	20.990	111.80	1362.0	17.060	
337	0.20480	161.10	0.060900	24.540	122.90	1873.0	18.770	
339	0.20890	202.40	0.141000	30.670	155.10	2906.0	23.510	
343	0.22550	157.60	0.110300	22.750	129.90	1540.0	19.680	
351	0.21350	119.40	0.124200	17.360	107.10	915.3	15.750	

426 rows × 9 columns

```
cv_data
```

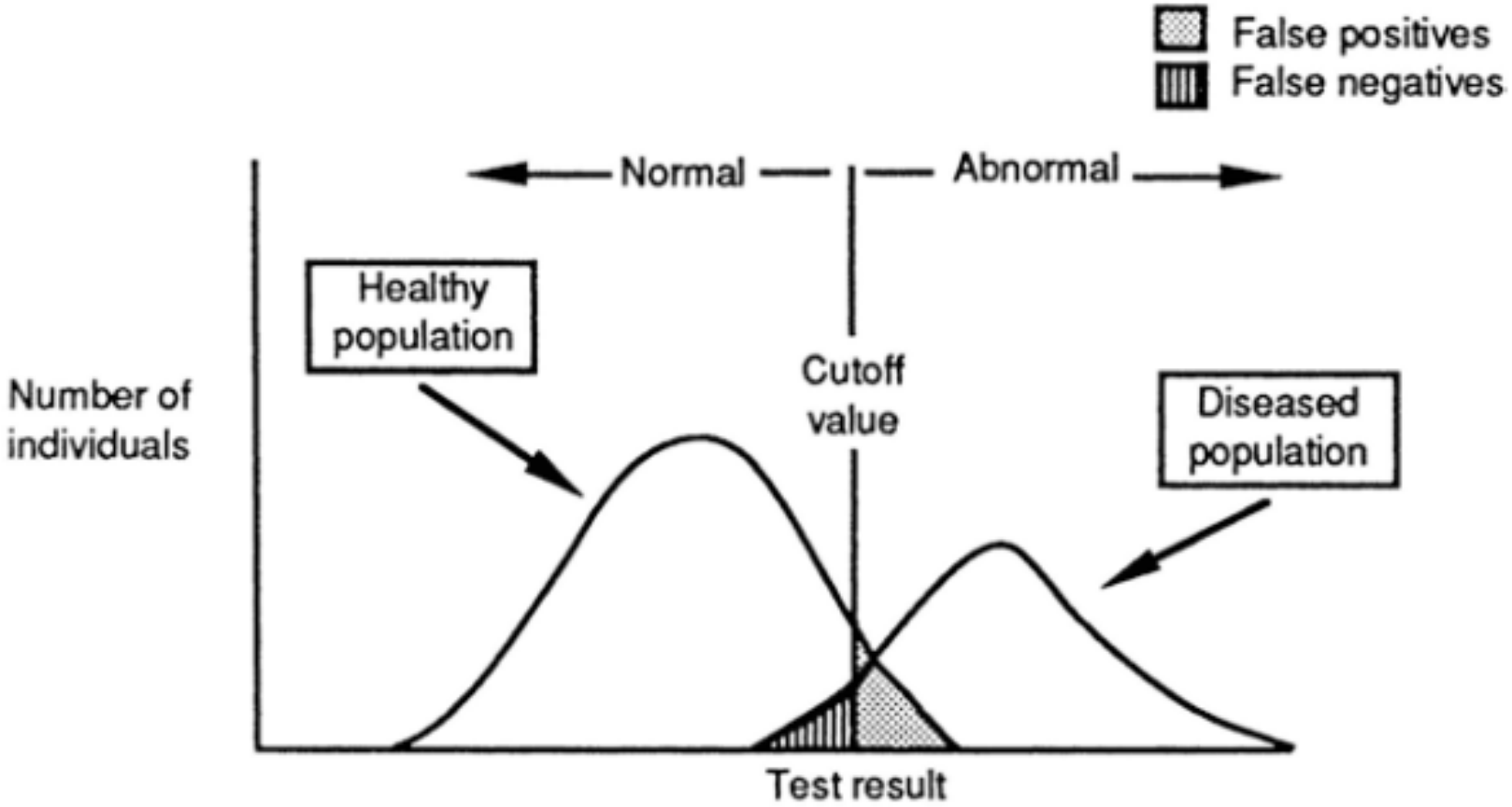
	concave points_worst	perimeter_worst	concave points_mean	radius_worst	perimeter_mean	area_worst	radius_mean	area
453	0.10690	103.10	0.06495	15.80	93.86	749.9	14.53	
454	0.09851	91.62	0.02272	14.34	80.62	633.5	12.62	
455	0.07763	96.69	0.03264	15.05	86.34	705.6	13.38	
456	0.06835	86.04	0.02017	13.12	74.87	527.8	11.63	
457	0.06005	91.29	0.02068	14.35	84.10	632.9	13.21	
...	...	...	...	...	...	...	...	
563	0.25420	179.10	0.14740	24.29	143.00	1819.0	20.92	
564	0.22160	166.10	0.13890	25.45	142.00	2027.0	21.56	
565	0.16280	155.00	0.09791	23.69	131.20	1731.0	20.13	
566	0.14180	126.70	0.05302	18.98	108.30	1124.0	16.60	
567	0.26500	184.60	0.15200	25.74	140.10	1821.0	20.60	

143 rows × 9 columns

Performance Evaluation

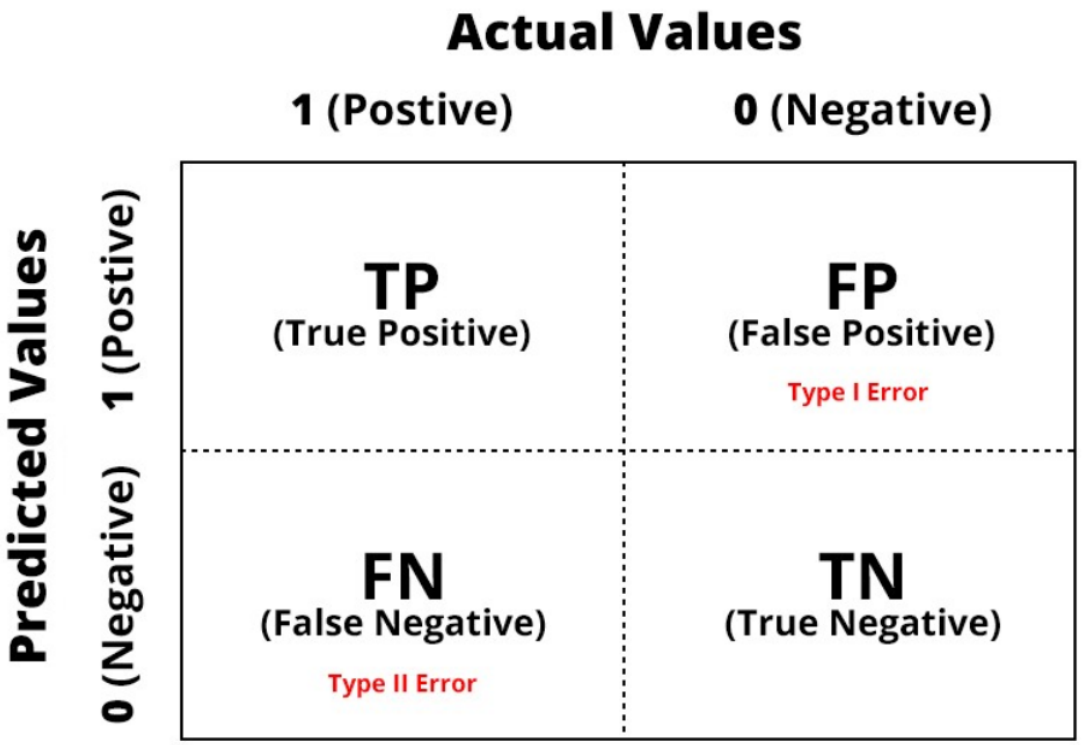
Classification of Test Results

Fig. 3.2 Distribution of test results in healthy and diseased individuals. Varying the cutoff between "normal" and "abnormal" across the continuous range of possible values changes the relative proportions of false positives (FPs) and false negatives (FNs) for the two populations



- **The two distributions (normal and abnormal) usually overlap.**

Confusion Matrix



Confusion Matrix

- A **true positive (TP) is a positive test result** obtained for a patient in whom the disease is present (the test result correctly classifies the patient as having the disease).
- A **true negative (TN) is a negative test result** obtained for a patient in whom the disease is absent (the test result correctly classifies the patient as not having the disease).
- A **false positive (FP) is a positive test result** obtained for a patient in whom the disease is absent (the test result incorrectly classifies the patient as having the disease).
- A **false negative (FN) is a negative test result** obtained for a patient in whom the disease is present (the test result incorrectly classifies the patient as not having the disease).

Measures of Test Performance (1)

- Measures of test performance are of two types: measures of agreement between tests or **measures of concordance**, and measures of disagreement or **measures of discordance**.
- **Ex :** in 2x2 table **concordant test results are TP and TN**.
- The **true-positive rate (TPR), or sensitivity**, is the likelihood that a diseased patient has a positive test.

$$\mathrm{p}[\text{positive test} \mid \text{disease}].$$

$$\mathrm{TPR} = \left(\frac{\text{number of diseased patients with positive test}}{\text{total number of diseased patients}} \right).$$

$$\mathrm{TPR} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}.$$

Measures of Test Performance (2)

- The **true-negative rate (TNR), or specificity,** is the likelihood that a nondiseased patient has a negative test result.

$$p[\text{negative test} \mid \text{no disease}].$$

$$\text{TNR} = \left(\frac{\text{Number of nondiseased patients with negative test}}{\text{Total number of nondiseased patients}} \right).$$

$$\text{TNR} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

Measures of Test Performance (3)

$$\text{FNR} = \left(\frac{\text{Number of diseased patients with negative test}}{\text{Total number of diseased patients}} \right) = \frac{\text{FN}}{\text{FN} + \text{TP}}.$$

$$\text{FPR} = \left(\frac{\text{Number of nondiseased patients with positive test}}{\text{Total number of nondiseased patients}} \right) = \frac{\text{FP}}{\text{FP} + \text{TN}}.$$

- The measures of discordance—the **false- positive rate (FPR) and the false-negative rate** (FNR).
- The **FNR** is the likelihood that a diseased patient has a negative test result.
- The **FPR** is the likelihood that a nondiseased patient has a positive test result.

Measures of Test Performance (4)

Example :

Consider the problem of screening blood donors for HIV. One test used to screen blood donors for HIV antibody is an enzyme-linked immunoassay (EIA). So that the performance of the EIA can be measured, the test is performed on 400 patients; the hypothetical results are shown in the 2 × 2 table :

Table 3.4 A 2×2 contingency table for HIV antibody EIA

EIA test result	Antibody present	Antibody absent	Total
Positive EIA	98	3	101
Negative EIA	2	297	299
	100	300	

EIA enzyme-linked immunoassay

Determine test performance !

Key Metrics:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

Measures of Test Performance (5)

- To determine test performance, we calculate the TPR (sensitivity) and TNR (specificitv) of the EIA antibodv test.

$$\text{TPR} = \frac{TP}{TP+FN} = \frac{98}{98+2} = 0.98.$$

- The likelihood that a patient with the HIV antibody will have a positive EIA test is 0.98.Conversely, we would expect the patients to receive incorrect, Negative results, for an **FNR** of 2%. (TPR + FNR = 1)
- And the **TNR** is : $\frac{TN}{TN+FP} = \frac{297}{297+3} = 0.99$
- The likelihood that a patient who has no HIV antibody will have a negative test is 0.99. Then, FPR = 0.01 (TNR + FPR = 1).

Performance Evaluation (1)

```
confusion_matrix(y_true=cv_data['diagnosis'],y_pred=predicted_classes)
```

```
array([[86,  4],
       [ 3, 50]], dtype=int64)
```

- **The number of True Positives (Tumors that were actually Malignant as well as classified as Malignant) = 86**
- **The number of False Negatives (Tumors that were actually Malignant but classified as Benign) = 4**
- **The number of False Positives (Tumors that were actually Benign but classified as Malignant) = 3**
- **The number of True Negatives (Tumors that were actually Benign as well as classified as Benign) = 50.**

Performance Evaluation (2)

```
print(classification_report(y_true=cv_data['diagnosis'],y_pred=predicted_classes))
```

```
              precision    recall  f1-score   support

           0       0.97      0.96      0.96        90
           1       0.93      0.94      0.93        53

    accuracy                           0.95       143
   macro avg       0.95      0.95      0.95       143
weighted avg       0.95      0.95      0.95       143
```

Key Metrics:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

Thank you